NO PRUMO

Rodando o NoPrumo na sua máquina

Setup do ambiente local: API em .NET 8, front em React e banco MySQL.

Este guia leva do repositório recém-clonado até a tela de login funcionando. São cinco passos, e só o segundo precisa de atenção: é o único que cada pessoa faz de um jeito diferente.

Atualizado em 17/09/2026.

Por que existe um passo de configuração

O arquivo appsettings.json vai para o GitHub junto com o código. Se a senha do banco estivesse nele, ela estaria publicada para sempre — e apagar depois não resolve, porque o histórico do git guarda tudo que já foi commitado uma vez.

Por isso as credenciais do projeto ficam no User Secrets, um recurso do .NET que guarda a configuração fora da pasta do repositório, dentro do seu perfil de usuário do Windows:

```
%APPDATA%MicrosoftUserSecretsd49dc0c7-694c-4e93-9ab0-940743b40311secrets.json
```

Esse número é o UserSecretsId, declarado no arquivo 01-Presentation.csproj. O identificador vai no commit; o conteúdo, não — ele está fora do repositório e o git sequer o enxerga.

> **É POR ISSO QUE CADA MÁQUINA CONFIGURA**
> Não é que o arquivo deixou de ser enviado por descuido: ele nunca poderia ser enviado. E isso é desejável — a senha do MySQL de cada pessoa é diferente, e a chave que assina os tokens não deve ser compartilhada entre máquinas.

Um detalhe que costuma confundir: o ASP.NET só lê o User Secrets em ambiente de Desenvolvimento. Em produção a configuração vem de variáveis de ambiente ou de um cofre de segredos. O mecanismo é outro de propósito.

O que precisa estar instalado

FERRAMENTA	VERSÃO	PARA QUÊ
.NET SDK	8.0	Compilar e rodar a API
MySQL Server	8.0	O banco de dados

Node.js	20 ou superior	Rodar o front (React + Vite)
Git	qualquer	Clonar e versionar

O MySQL Workbench é opcional, mas ajuda a olhar o banco quando algo não bate com a tela.

Passo 1 — Clonar o repositório

```
NO PROMPT DE COMANDO
git clone https://github.com/GabrielHenriqueCe/NoPrumo.git
cd NoPrumo
```

Passo 2 — Configurar os segredos

São quatro chaves. Antes delas, gere a chave que assina os tokens:

```
GERAR A CHAVE DO JWT — É UM COMANDO SÓ, EM UMA LINHA
powershell -Command "$b=New-Object byte[] 64;
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b);
[Convert]::ToBase64String($b)"
```

Copie o texto que aparecer e use no lugar de A_CHAVE_GERADA abaixo. Rode os quatro comandos na raiz do repositório:

```
CADA COMANDO É UMA LINHA SÓ — A PRIMEIRA ESTÁ QUEBRADA AQUI APENAS PARA CABER NA PÁGINA
dotnet user-secrets set "ConnectionStrings:DefaultConnection"
  "Server=localhost;Port=3306;Database=no_prumo;User=root;Password=SUA_SENHA"
  --project back/01-Presentation
dotnet user-secrets set "Jwt:Key" "A_CHAVE_GERADA" --project back/01-Presentation
dotnet user-secrets set "Jwt:Issuer" "NoPrumo" --project back/01-Presentation
dotnet user-secrets set "Jwt:Audience" "NoPrumo" --project back/01-Presentation
```

Confira no final. O comando abaixo tem que listar as quatro chaves:

```
dotnet user-secrets list --project back/01-Presentation
```

- Cada pessoa gera a própria chave do JWT. O token é assinado e conferido na mesma máquina, então não precisa ser igual entre vocês.
- Issuer e Audience podem ser qualquer texto, desde que iguais entre si dentro da mesma máquina — é a API conferindo o que ela mesma emitiu.
- Trocar a chave invalida os tokens já emitidos. Basta sair e entrar de novo.
- Sem as quatro chaves a API não sobe: o Program.cs lança exceção de propósito. Falhar no start é melhor do que rodar com assinatura inválida.

Passo 3 — Criar o banco

A ferramenta de linha de comando do Entity Framework é instalada uma vez por máquina, não por projeto:

```
O SEGUNDO COMANDO É UMA LINHA SÓ
dotnet tool install --global dotnet-ef
dotnet ef database update --project back/03-Infrastructure
  --startup-project back/01-Presentation
```

A migration cria as 37 tabelas vazias. Quem cria as permissões, os papéis e o primeiro usuário é o seeder, que roda sozinho quando a API sobe — e só insere o que estiver faltando, então rodar dez vezes dá no mesmo que rodar uma.

> **NUNCA EDITE UM ARQUIVO DE MIGRATION**
> As migrations são geradas exclusivamente pelo comando do EF, sem uma linha de SQL escrita à mão. Se alguma coisa precisa existir no banco — um valor padrão, uma restrição, um seed — declare no AppDbContext e gere a migration pelo comando.

Passo 4 — Subir a API e o front

O jeito rápido: dois cliques no arquivo dev.bat, na raiz do repositório. Ele abre uma janela com a API, outra com o front, e chama o navegador na tela de login.

Na mão, são duas janelas do prompt, uma para cada:

```
JANELA 1 — API, PORTA 5262
dotnet run --project back/01-Presentation --launch-profile http
```

```
JANELA 2 — FRONT, PORTA 5173
cd front
npm install
npm run dev
```

O npm install só é necessário na primeira vez. Cada janela fica ocupada enquanto o serviço estiver de pé; Ctrl+C para parar.

Uma diferença que vale saber: o front recarrega sozinho quando você salva um arquivo. A API não — ela compila uma vez ao subir, então mudou C#, é Ctrl+C e subir de novo.

Passo 5 — Entrar

Abra http://localhost:5173 e entre com:

CAMPO	VALOR
Usuário	admin
Senha	admin

O sistema obriga a trocar a senha no primeiro acesso. Não existe auto-cadastro em lugar nenhum: o administrador cria as contas e entrega uma senha provisória, que a pessoa troca ao entrar.

A documentação da API, com todos os endpoints, fica em http://localhost:5262/swagger. O botão Authorize, no topo, aceita o token devolvido pelo login e libera testar os endpoints protegidos.

Quando der errado

SINTOMA	CAUSA PROVÁVEL E SOLUÇÃO
A API não sobe e fala em chaves Jwt	Faltou o passo 2. Rode dotnet user-secrets list e confira se as quatro chaves aparecem.
Erro de conexão com o banco	MySQL parado, ou senha errada na connection string. Confira o serviço no Windows e refaça o primeiro comando do passo 2.
Porta 5173 ou 5262 ocupada	Sobrou um processo de antes. Use netstat -ano | findstr :5173 para achar o PID, e taskkill /f /pid NUMERO para encerrar.
Login responde 401 com admin/admin	Alguém já trocou a senha nesse banco. Apague a linha da tabela user e suba a API de novo: o seeder recria o admin.
A tela abre mas nada carrega	A API está parada, ou o endereço mudou. O front usa VITE_API_URL, com http://localhost:5262/api como padrão.

Invalid token depois de trocar a chave	Esperado: o token antigo foi assinado com a chave anterior. Saia e entre de novo.